Chapter V – Supervised Learning

Instituto Superior de Engenharia
Politécnico de Coimbra

Simão Paredes
sparedes@isec.pt

Outline

- Learning Process
- Validation Strategies
- Algorithms:
  - Linear Regression
  - Logistic Regression
  - Decision Trees
  - Support Vector Machine
  - Naïve Bayes
  - KNN-nearest Neighbour
  - Random Forest
  - Neural Networks

Learning Process

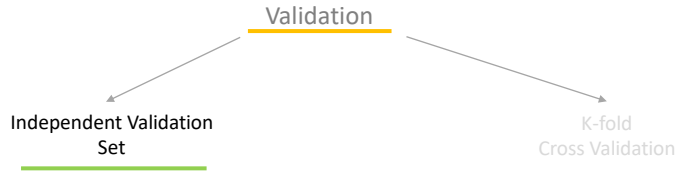
Learning Process

- A learning process, model creation, may be decomposed in three main elements:

Model (algorithm)	A way to explain the relationships between data, i.e. to translate input instances into outputs.	**Linear Regression** **SVM** **ANN** **DT** **…**
Cost Function	It measures how close the model performance is to its intended result.	**SSE** **MSE; RMSE** **Cross-Entropy** **…**
Minimization of the Cost Function / Error	Based on data, an iterative procedure (e.g optimization) is carried out in order to minimize the cost function/error	**Least Square Method** **Gradient descendent** **Stochastic gradient descendent** **Grid Search** **Back propagation** **…**

Validation Strategy

Adopted Validation Strategy




```
# split data
data = ...
train, validation, test = split(data)

# model
parameters = ...
model = fit(train)

# tune model hyperparameters
for params in parameters:
model = fit(train, params)
skill = evaluate(model, validation)

# evaluate final model
model = fit(train)
skill = evaluate(model, test)
```

https://machinelearningmastery.com/difference-test-validation-datasets/

Limitations:

- It is a single evaluation of the model and **might have limited ability** to characterize the uncertainty in the results.
- With **small sample sizes**:
  - the model may need every possible instance to learn the model parameters.
  - Different validation **sets may produce very different results.**
  - Resampling methods can produce reasonable predictions of how well the model will perform on future samples.

Applied Predictive Modeling 2nd ed. 2018, Corr. 2nd printing 2018 Edition

Adopted Validation Strategy



Actually, k-fold cross-validation within the training dataset, is the most frequent current approach.

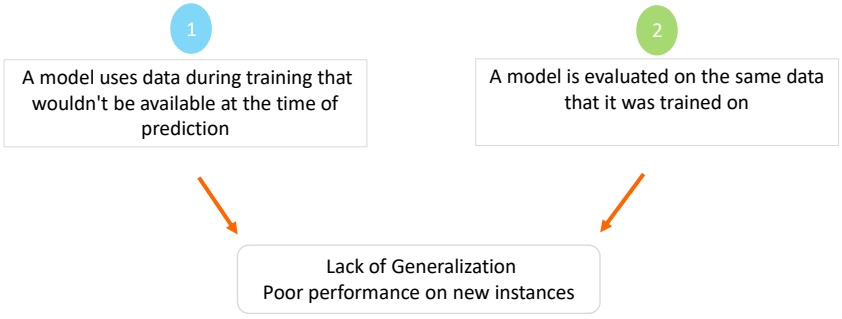
In some cases, small/medium size datasets, cross-validation may be applied to the entire dataset.

```
# split data
data = ...
train, test = split(data)
# model
parameters = ...
model = fit(train)
# tune model hyperparameters
for params in parameters:
    skills = list()
    for i in k:
        fold_train, fold_val = cv_split(i, k, train)
        model = fit(fold_train, params)
        skill_estimate = evaluate(model, fold_val)
        skills.append(skill_estimate)
    skill = summarize(skills)
# evaluate final model
model = fit(train)
skill = evaluate(model, test)
```


Data Leakage

Data Leakage

- In machine learning context, data leakage occurs when:



  - Actually, the goal of predictive modeling is to develop a machine learning model capable of making accurate predictions on real world future data that is not available during training
    - However, when data leakage occurs, the model performs well in testing and validation, but when used in production, it becomes entirely inaccurate **(lack of generalization)**.

Data Leakage

- Two types of data leakage:

Target leakage | Train test contamination

  - Target leakage
    - Models include data that will not be available when the model is used to make predictions.
      - Example: Model to predict credit card fraud.
        - This is a forecast application where models must make reliable future predictions based on incomplete data.
        - The raw dataset contains information about the customer, the transaction amount, the location, **whether fraud was detected (label) and if a chargeback was received**.

Data Leakage

- Target leakage
  - During the training of the model, the **fraud detection** and **chargeback** have the respective values entered (Yes/No).

	amount	location	...	chargeback	fraud detection
1	**	**	**	Yes	Yes
...					

  - In the real world, a **chargeback is typically initiated after fraud has been detected**, so this **information would not be available** at the time of detection.
  - Training a model with this data "teaches" it that transactions with a chargeback are almost always fraudulent.
    - During validation (test set), the model may present high performance because, in training, the relationship between fraud and chargeback is strong.
    - However, the chargeback information will not be available in the deployment phase. So, the model will perform poorly in practice and will be useless.

Data Leakage

- Train Test contamination
  - It occurs when both training and test data are used to create a model, often due to improper splitting or preprocessing.
    - Example: Build a model to predict house prices based on features such as house size, number of bedrooms and neighborhood.
      - To assure that all the numerical features have the same scale a standard scaler is applied.
        - If this standardization is applied to the **entire dataset** before splitting it into training and test datasets, the model will indirectly "see" information from the test set during training.
        - This probably makes it easier for the model to perform well on the test set but potentially reduces its ability to generalize to new, unseen data.

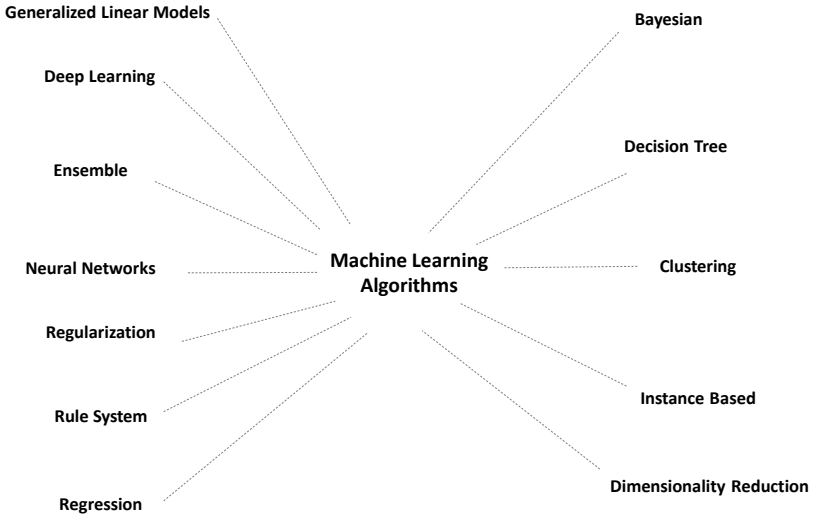
Data Leakage

- Train test contamination
  - Preprocessing steps such as scaling, imputation or feature selection **should be fitted only on the training data** and then applied to the test set, rather than fitting them on the entire dataset before splitting.
    - A incorrect application of transformers such as scaling or normalization can lead to train test contamination.
    - When these preprocessing steps are performed **over the whole dataset**, it may lead to biased predictions and an unrealistic sense of the model's performance.

Regression

Supervised Learning Algorithms

Machine Learning Algorithms

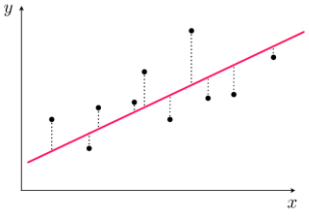


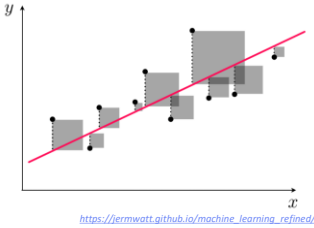
1 Linear Regression
Polynomial Regression
Logistic Regression * (**binary classification**)
LOESS (local regression), …

Adapted from: https://machinelearningmastery.com/classification-and-regression-trees-for-machine-learning/

Machine Learning Algorithms

- The learning process varies from model to model but the goal is always the same:
  - The model learns how to take X (i.e. features) in order to predict y (i.e. target)
  - The objective of training a ML model, is to **find parameters, weights or a structure** that **minimizes the cost function/error.**





e.g. **Least squares**: Minimization of the grey areas

https://jermwatt.github.io/machine_learning_refined/notes/5_Linear_regression/5_2_Least.html

"… a cost function is a measure of how wrong the model is in terms of its ability to estimate the relationship between X and y"

Linear Regression

Linear Regression

- A linear model is based on a weighted sum of inputs (features values) plus a constant (bias term/intercept term) such as:

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

where:

- $\hat{y}$ is the estimated value of y
- n number of features
- x_i is the ith feature
- θ_j is the jth model parameter (θ_0 the bias term, $\theta_1 \ldots \theta_n$ the features weights) *

* In the specific context of regression models coefficients might also be presented using $\boldsymbol{\beta}_j$, however by convention the letter $\boldsymbol{\theta}$ is frequently applied to represent ML model parameters

Linear Regression

- It can be rewritten in a more compact way:

$$\hat{y} = h_\theta(\mathbf{x}) = \boldsymbol{\theta} \cdot \mathbf{x}$$

where:

- $\boldsymbol{\theta}$ is the model parameter vector (the bias term θ_0 and the features weights $\theta_1 \ldots \theta_n$)
- $\mathbf{x}$ is the instance's feature vector ($x_0 \ldots x_n;\ x_0 = 1$)
- $\boldsymbol{\theta} \cdot \mathbf{x}$ is the dot product of two vectors ($\sum_{i=0}^{n} \theta_i x_i$)

* In Machine Learning vectors are often represented as column vectors (2D, single column), if $\boldsymbol{\theta}$ and $\mathbf{x}$ are column vectors, the dot product can be given as a matrix multiplication $\boldsymbol{\theta}^T \mathbf{x}$

- h_θ is the hypothesis function using the model parameters $\boldsymbol{\theta}$

Linear Regression

- Single independent variable:
  - The coefficient defines **how much the dependent variable is expected to increase** (if the coefficient is positive) **or decrease** (if the coefficient is negative) **when that independent variable increases by one.**

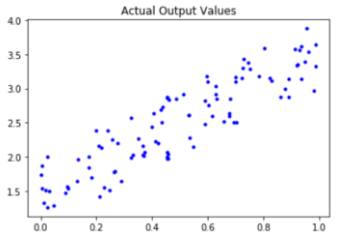
$$\hat{y} = \theta_0 + \theta_1 x_1$$

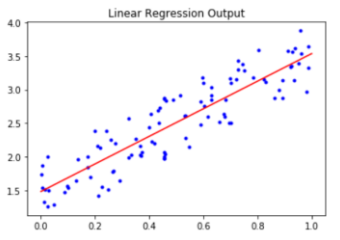
- Multiple independent variables:
  - The coefficient indicates **how much the dependent variable is expected to increase/descrease when that independent variable increases by one,** holding all the other independent variables constant.

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

Linear Regression

- How to train the model?
  - Set its parameters $\boldsymbol{\theta}$ so that the model best fits the training set





?

  - The training phase seeks to minimize a cost function
    - Find the values of $\boldsymbol{\theta}$ that minimizes a cost function (e.g. MSE) considering m instances

$$\text{MSE}(\mathbf{X}, h) = \frac{1}{m}\sum_{i=1}^{m}(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)})^2 = \frac{1}{m}\sum_{i=1}^{m}(\boldsymbol{\theta}^T\mathbf{x}^{(i)} - y^{(i)})^2$$

Linear Regression

- Training phase
  - Minimize a cost function:

$$\text{MSE}(\mathbf{X}, h) = \frac{1}{m}\sum_{i=1}^{m}(\boldsymbol{\theta}^T\mathbf{x}^{(i)} - y^{(i)})^2$$

  - Approach to find $\boldsymbol{\theta}$?
    - **Gradient Descent !!**
      - Optimization method!

Linear Regression

- Gradient Descent
  - The gradient of the cost function is computed regarding each model parameter θ_j
    - The algorithm has 5 steps:

1. Initialize the weights $\boldsymbol{\theta}$ with random values
   Calculate Error (e.g. MSE)
2. Calculate the Gradient:
   Change in MSE when the weights $\boldsymbol{\theta}$ are modified from their randomly initialized values
3. Adjust the weights in order to minimize MSE
   $$\boldsymbol{\theta}^{(new)} = \boldsymbol{\theta}^{(old)} - \eta \nabla_{\boldsymbol{\theta}} MSE(\boldsymbol{\theta})$$
4. Use the new weights $\boldsymbol{\theta}^{(new)}$ for prediction and to calculate the new MSE
5. Repeat steps 2 - 4 until further adjustments to weights don't significantly reduce the Error

Gradient Descent (example)

- Considering a linear model:

$$h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

- where $\boldsymbol{\theta} = \begin{pmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \end{pmatrix}$ are the model parameters and $\mathbf{x} = \begin{pmatrix} 1 \\ x_1 \\ x_2 \end{pmatrix}$ the feature vector with $x_o = 1$ for the intercept

- A very simple dataset (only 2 instances):

Instance i	$x_{i,1}$	$x_{i,2}$	y_i
1	2	3	10
2	4	1	15

Gradient Descent (example)

- Cost Function:

$$J(\boldsymbol{\theta}) = \frac{1}{2m}\sum_{i=1}^{m}(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)})^2$$

$\frac{1}{2m}$ only for mathematical convenience as the derivative is simpler

- Gradient Descent Update Rule

$$\theta_j^{(z)} = \theta_j^{(z-1)} - \alpha\frac{\partial}{\partial\theta_j}J(\boldsymbol{\theta}) \qquad z - iteration$$

  - The partial derivative (gradient component) is given by:

$$\frac{\partial}{\partial\theta_j}J(\boldsymbol{\theta}) = \frac{\mathbf{1}}{\boldsymbol{m}}\sum_{i=1}^{m}(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)}) \ .x_{i,j}$$

    - $j = 0$ for the intercept θ_0 where $x_{i,0} = 1$
    - $j = 1$ for the first feature θ_1 where $x_{i,1}$ is the value of the first feature of the i instance
    - $j = 2$ for the second feature θ_2 where $x_{i,2}$ is the value of the second feature of the i instance

Gradient Descent (example)

- Initial Values and Hyperparameters:
  - Initial parameters:
    - $\theta_0 = 0;\ \theta_1 = 0;\ \theta_2 = 0$
  - Learning rate (α):
    - $\alpha = 0.01$ (a common starting value)
  - Number of instances (m):
    - $m = 2$

$$\theta_j^{(z)} = \theta_j^{(z-1)} - \alpha\frac{\partial}{\partial\theta_j}J(\boldsymbol{\theta})$$

Instance i	$x_{i,1}$	$x_{i,2}$	y_i
1	2	3	10
2	4	1	15

- Iterations
  - Iteration **1**
    - $\boldsymbol{\theta}^{(0)} = \begin{pmatrix}0\\0\\0\end{pmatrix}$

Step 1
Calculate the predictions

$$h_{\boldsymbol{\theta}}(\mathbf{x}^{(1)}) = 0 + 0\times 2 + 0\times 3 = 0$$
$$h_{\boldsymbol{\theta}}(\mathbf{x}^{(2)}) = 0 + 0\times 4 + 0\times 1 = 0$$

Gradient Descent (example)

- Iteration **1**

Step 2
Calculate Errors

$Instance\ i$	$x_{i,1}$	$x_{i,2}$	y_i
1	2	3	10
2	4	1	15

$$h_{\boldsymbol{\theta}}(\mathbf{x}^{(1)}) - y^{(1)} = 0 - 10 = -10$$
$$h_{\boldsymbol{\theta}}(\mathbf{x}^{(2)}) - y^{(2)} = 0 - 15 = -15$$

Step 3
Calculate the Gradient $\nabla J(\boldsymbol{\theta}^{(0)})$

$$\frac{\partial}{\partial \theta_j} J(\boldsymbol{\theta}) = \frac{\mathbf{1}}{\boldsymbol{m}} \sum_{i=1}^{m} (h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)}) \cdot x_{i,j}$$

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{2}[-10 \times 1 + (-15) \times 1] = -12.5$$
$$\frac{\partial J}{\partial \theta_1} = \frac{1}{2}[-10 \times 2 + (-15) \times 4] = -40$$
$$\frac{\partial J}{\partial \theta_2} = \frac{1}{2}[-10 \times 3 + (-15) \times 1] = -22.5$$

Gradient Descent (example)

- Iteration **1**

Step 4
Update Parameters

$$\theta_0^{(1)} = 0 - 0.01 \times (-12.5) = 0.125$$
$$\theta_1^{(1)} = 0 - 0.01 \times (-40) = 0.4$$
$$\theta_2^{(1)} = 0 - 0.01 \times (-22.5) = 0.225$$

$$\boldsymbol{\theta}^{(0)} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix} \rightarrow \boldsymbol{\theta}^{(1)} = \begin{pmatrix} 0.125 \\ 0.4 \\ 0.225 \end{pmatrix}$$

Gradient Descent (example)

- Iteration **2**

- $\boldsymbol{\theta}^{(1)} = \begin{pmatrix} 0.125 \\ 0.4 \\ 0.225 \end{pmatrix}$

$Instance\ i$	$x_{i,1}$	$x_{i,2}$	y_i
1	2	3	10
2	4	1	15

Step 1
Calculate the predictions

$$h_{\boldsymbol{\theta}}(\mathbf{x}^{(1)}) = 0.125 + 0.4 \times 2 + 0.225 \times 3 = 1.6$$
$$h_{\boldsymbol{\theta}}(\mathbf{x}^{(2)}) = 0.125 + 0.4 \times 4 + 0.225 \times 1 = 1.95$$

Step 2
Calculate Errors

$$h_{\boldsymbol{\theta}}(\mathbf{x}^{(1)}) - y^{(1)} = 1.6 - 10 = -8.4$$
$$h_{\boldsymbol{\theta}}(\mathbf{x}^{(2)}) - y^{(2)} = 1.95 - 15 = -13.05$$

Gradient Descent (example)

- Iteration **2**

$Instance\ i$	$x_{i,1}$	$x_{i,2}$	y_i
1	2	3	10
2	4	1	15

Step 3
Calculate the Gradient $\nabla J(\boldsymbol{\theta}^{(0)})$

$$\frac{\partial}{\partial \theta_j} J(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^{m} (h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)}) \cdot x_{i,j}$$

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{2}[-8.4 \times 1 + (-13.05) \times 1] = -10.725$$
$$\frac{\partial J}{\partial \theta_1} = \frac{1}{2}[-8.4 \times 2 + (-13.05) \times 4] = -34.5$$
$$\frac{\partial J}{\partial \theta_2} = \frac{1}{2}[-8.4 \times 3 + (-13.05) \times 1] = -19.125$$

Gradient Descent (example)

- Iteration **2**

Step 4
Update Parameters

$$\theta_0^{(2)} = 0.125 - 0.01 \times (-10.725) = 0.232$$

$$\theta_1^{(2)} = 0.4 - 0.01 \times (-34.5) = 0.745$$

$$\theta_2^{(2)} = 0.225 - 0.01 \times (-19.125) = 0.416$$

$$\boldsymbol{\theta}^{(0)} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix} \rightarrow \boldsymbol{\theta}^{(1)} = \begin{pmatrix} 0.125 \\ 0.4 \\ 0.225 \end{pmatrix} \rightarrow \boldsymbol{\theta}^{(2)} = \begin{pmatrix} 0.232 \\ 0.745 \\ 0.416 \end{pmatrix}$$

Gradient Descent (example)

- Iteration **3**

- $\boldsymbol{\theta}^{(2)} = \begin{pmatrix} 0.232 \\ 0.745 \\ 0.416 \end{pmatrix}$

$Instance\ i$	$x_{i,1}$	$x_{i,2}$	y_i
1	2	3	10
2	4	1	15

Step 1
Calculate the predictions

$$h_{\boldsymbol{\theta}}(\mathbf{x}^{(1)}) = 0.232 + 0.745 \times 2 + 0.416 \times 3 = 2.97$$
$$h_{\boldsymbol{\theta}}(\mathbf{x}^{(2)}) = 0.232 + 0.745 \times 4 + 0.416 \times 1 = 3.628$$

Step 2
Calculate Errors

$$h_{\boldsymbol{\theta}}(\mathbf{x}^{(1)}) - y^{(1)} = 2.97 - 10 = -7.03$$
$$h_{\boldsymbol{\theta}}(\mathbf{x}^{(2)}) - y^{(2)} = 3.628 - 15 = -11.372$$

Gradient Descent (example)

- Iteration **3**

Instance i	$x_{i,1}$	$x_{i,2}$	y_i
1	2	3	10
2	4	1	15

Step 3
Calculate the Gradient $\nabla J(\boldsymbol{\theta}^{(0)})$

$$\frac{\partial}{\partial \theta_j} J(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^{m} (h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)}) \cdot x_{i,j}$$

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{2}[-7.030 \times 1 + (-11.372) \times 1] = -9.201$$

$$\frac{\partial J}{\partial \theta_1} = \frac{1}{2}[-7.030 \times 2 + (-11.372) \times 4] = -29.774$$

$$\frac{\partial J}{\partial \theta_2} = \frac{1}{2}[-7.030 \times 3 + (-11.372) \times 1] = -16.231$$

Gradient Descent (example)

- Iteration **3**

Step 4
Update Parameters

$$\theta_0^{(2)} = 0.232 - 0.01 \times (-9.201) = 0.324$$

$$\theta_1^{(2)} = 0.745 - 0.01 \times (-29.774) = 1.043$$

$$\theta_2^{(2)} = 0.416 - 0.01 \times (-16.231) = 0.579$$

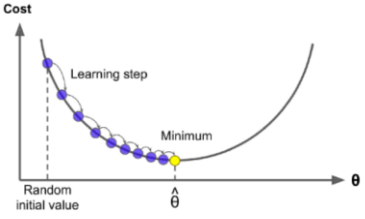
$$\boldsymbol{\theta}^{(0)} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix} \rightarrow \boldsymbol{\theta}^{(1)} = \begin{pmatrix} 0.125 \\ 0.4 \\ 0.225 \end{pmatrix} \rightarrow \boldsymbol{\theta}^{(2)} = \begin{pmatrix} 0.232 \\ 0.745 \\ 0.416 \end{pmatrix} \rightarrow \boldsymbol{\theta}^{(3)} = \begin{pmatrix} 0.324 \\ 1.043 \\ 0.579 \end{pmatrix}$$

$$\mathbf{err}^{(1)} = \begin{pmatrix} -10 \\ -15 \end{pmatrix} \rightarrow \mathbf{err}^{(2)} = \begin{pmatrix} -8.4 \\ -13.5 \end{pmatrix} \rightarrow \mathbf{err}^{(3)} = \begin{pmatrix} -7.03 \\ -11.372 \end{pmatrix}$$

Linear Regression

- Gradient Descent
  - Generic optimization algorithm able to find an optimal solution for several problems,
    - the goal of Gradient Descendent is to adjust parameters $\boldsymbol{\theta}$, **iteratively**, in order to minimize a cost function

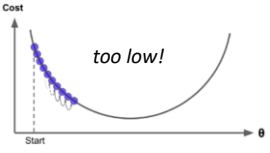
$$\nabla_{\boldsymbol{\theta}} MSE(\boldsymbol{\theta}) = \begin{pmatrix} \frac{\partial}{\partial \theta_0} MSE(\boldsymbol{\theta}) \\ \frac{\partial}{\partial \theta_1} MSE(\boldsymbol{\theta}) \\ \dots \\ \frac{\partial}{\partial \theta_n} MSE(\boldsymbol{\theta}) \end{pmatrix} = \frac{2}{m} \mathbf{X}^T (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})$$



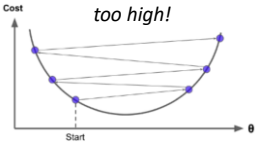
    - It goes in the direction of descendent gradient, until reach the value zero.
    - Once the gradient is zero, a minimum of the cost function has been found.

Linear Regression

- Learning Rate of Gradient Descent
  - In the learning process, update of parameters, learning rate is a **very important hyperparameter**
    - If the **learning rate is too low**
      - The algorithm will have many iterations (long time to converge)

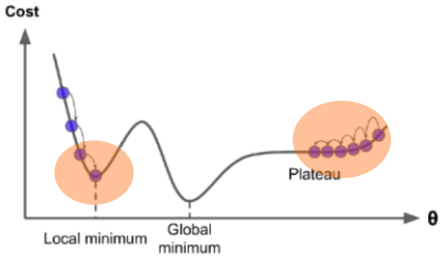


    - if the **learning rate is too high**
      - The algorithm may diverge as it can jump over the minimum



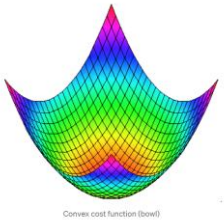
Linear Regression

- Gradient Descent
  - The irregular shape of a cost function, may represent an additional difficulty for the operation of Gradient Descent, as:
    - It can converge to a local minimum or being stuck in a plateau



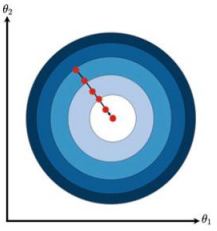
Linear Regression

- Gradient Descent
  - However, MSE has some specific characteristics that avoid these drawbacks:
    - If the learning rate is properly set, **Gradient Descent will reach the global minimum** of MSE, as:
      - MSE is a convex function
        - A line joining two points on the curve never crosses the curve
      - MSE has no local minima, just one global minimum
      - MSE is a continuous function with a slope that never changes abruptly

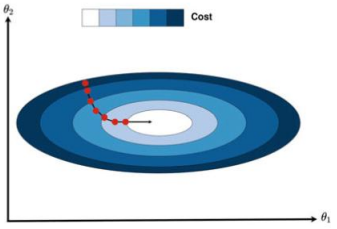


Linear Regression

- Gradient Descent
  - The **training model process:**
    - It searches for a combination of model parameters $\hat{\boldsymbol{\theta}}$ that minimizes the cost function
      - Over the training set: some / all instances).
    - The more parameters a model has (number of features) the harder the search is
    - With Gradient Descent all the **features should have a similar scale**
      - Otherwise it will take much longer to converge.



*Gradient descent **with** feature scaling*



*Gradient descent **without** feature scaling*

Linear Regression

- Gradient Descent
  - Three different approaches

Batch Gradient Descent

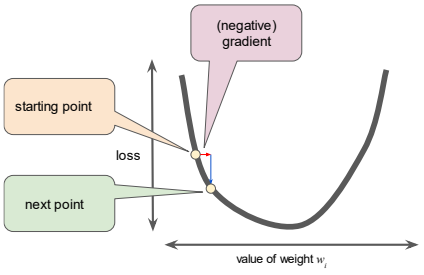
Stochastic (random) Gradient Descent

Mini-Batch Gradient Descendent

Linear Regression

- Batch Gradient Descent
  - This approach involves the calculation over the full training set $\mathbf{X}$

$$\nabla_{\boldsymbol{\theta}} MSE(\boldsymbol{\theta}) = \begin{pmatrix} \frac{\partial}{\partial \theta_0} MSE(\boldsymbol{\theta}) \\ \frac{\partial}{\partial \theta_1} MSE(\boldsymbol{\theta}) \\ \dots \\ \frac{\partial}{\partial \theta_n} MSE(\boldsymbol{\theta}) \end{pmatrix} = \frac{2}{m} \mathbf{X}^T (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})$$



  - At each descent gradient step:
    - The m instances that correspond to the whole batch of training set are considered to calculate the cost function

Linear Regression

- Batch Gradient Descent

$$\nabla_{\boldsymbol{\theta}} MSE(\boldsymbol{\theta}) = \begin{pmatrix} \frac{\partial}{\partial \theta_0} MSE(\boldsymbol{\theta}) \\ \frac{\partial}{\partial \theta_1} MSE(\boldsymbol{\theta}) \\ \dots \\ \frac{\partial}{\partial \theta_n} MSE(\boldsymbol{\theta}) \end{pmatrix} = \frac{2}{m} \mathbf{X}^T (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})$$

  - After calculating $\nabla_{\boldsymbol{\theta}} MSE(\boldsymbol{\theta})$, the new/updated $\boldsymbol{\theta}$ with $n+1$ parameters (n features) is obtained considering the learning rate η through:

$$\boldsymbol{\theta}^{(next\ step)} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} MSE(\boldsymbol{\theta})$$

$$\nabla_{\boldsymbol{\theta}} MSE(\boldsymbol{\theta}) = \alpha \mathbf{X}^T_{(n+1)\times m} (\mathbf{X}_{m\times(n+1)} \boldsymbol{\theta}_{(n+1)\times 1} - \mathbf{y}_{m\times 1})$$

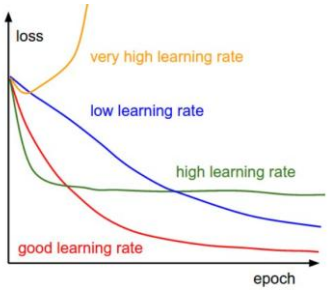
$m \times 1$

$(n+1) \times 1$

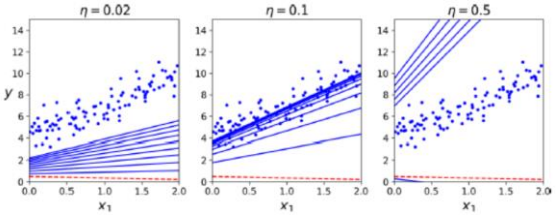
m instances
n features

Linear Regression

- Batch Gradient Descent
  - Learning rate is small:
    - Long time to converge
  - Learning rate is too high
    - It may diverge



https://towardsdatascience.com/gradient-descent-explained-9b953fc0d2c



Linear Regression

- Batch Gradient Descent

```
eta=0.1
n_iterations=1000
m=100

theta=np.random.randn(2,1)

for iteration in range(n_iterations):
        gradients=2/m *X_b.T.dot(X_b.dot(theta)-y)
        theta=theta-eta*gradients
```

$$\nabla_{\boldsymbol{\theta}} MSE(\boldsymbol{\theta}) = \alpha \mathbf{X}^T_{(n+1)\times m}(\mathbf{X}_{m\times(n+1)}\boldsymbol{\theta}_{(n+1)\times 1} - \mathbf{y}_{m\times 1})$$

```
array([[4.21638028],
       [2.6589212 ]])
```

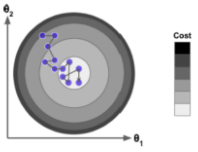
  - Main Drawback
    - It uses the **whole training set** to compute the gradients in every step which may make this optimization very slow
      - Mainly with large training sets / high dimensional spaces

Linear Regression

- Stochastic (random) Gradient Descent
  - It picks a random instance i in the training set and computes the gradient based only on the error of that instance.

$$\mathrm{E}(\mathbf{x}, h) = (\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)})^2$$

    - Advantage: Less data, much faster!
      - The dimension of dataset is not a problem as only one instance is considered at a time.
    - Disadvantages:
      - less regular than Batch Gradient Descent
        - the cost function will bounce up and down decreasing only on average.
      - The cost function is more irregular.



Linear Regression

- Mini-Batch Gradient Descendent
  - It is a middle term between Batch Gradient Descent and Stochastic Gradient Descent
    - It computes the gradient on a small random of instances designated as Mini-batch GD
    - It is less erratic that Stochastic GD and does not require the effort of Batch GD
    - Downsides:
      - An additional configuration of the hyper parameter “mini-batch size” for the learning algorithm is required.

Linear Regression

- Linear Regression:

Algorithm	Large m	Large n	Scaling Required	Scikit-Learn
Batch GD	Slow	Fast	Yes	SGDRegressor
Stochastic GD	Fast	Fast	Yes	SGDRegressor
Mini Batch GD	Fast	Fast	Yes	SGDRegressor

m – instances; n - features

  - These algorithms provide similar models

Similar Models $\Longrightarrow$ Similar Predictions

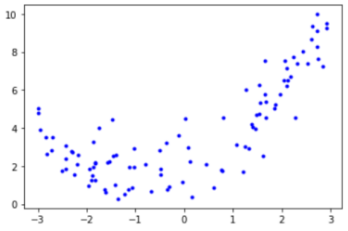
Polynomial Regression

Polynomial Regression

- If the data is nonlinear then a polynomial regression might be applied:

$$\hat{y} = \theta_0 + \theta_1 x \quad + \theta_2 x^2 \; + \cdots + \theta_k x^k$$

  - Data cannot be represented as a straight line, it requires a high order expression (non linear)



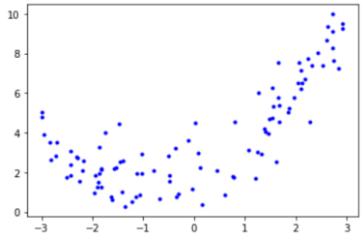
  - this type of regression can easily explode the complexity of the model, so the degree of the polynomial expression should be carefully controlled
  - this approach is also prone to overfitting situations

Polynomial Regression

- Non Linear (high order) expression

Example:

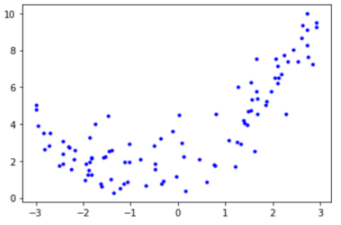
$y = 0.5x^2 + x + 2$




```
m=100
X=6*np.random.rand(m,1)-3                    #Uniform distribution
y=0.5 * X**2 + X + 2 + np.random.randn(m,1)  #Gaussian Distribution
plt.plot(X,y,'b.');
```

Polynomial Regression

- It is obvious that a straight line cannot fit the data



- Possible solution?
  - transform the training data, adding the square (second degree polynomial) of each feature (e.g in this case there is just one feature)

```
1 from sklearn.preprocessing import PolynomialFeatures
2 poly_features=PolynomialFeatures(degree=2, include_bias=False)
3 X_poly=poly_features.fit_transform(X)
4 print(X[0])
5 print(X_poly[0])
```

```
[2.11115026]
[2.11115026 4.45695541]
```


Polynomial Regression

- Fitting the model:

```
1 from sklearn.linear_model import LinearRegression
2 lin_reg=LinearRegression()
3 lin_reg.fit(X_poly,y)
```

- Coefficients: $y = \theta_0 + \theta_1 x_1 + \theta_2 x_1^2$

```
lin_reg.intercept_
```

```
array([1.95674409])
```

```
1 lin_reg.coef_
```

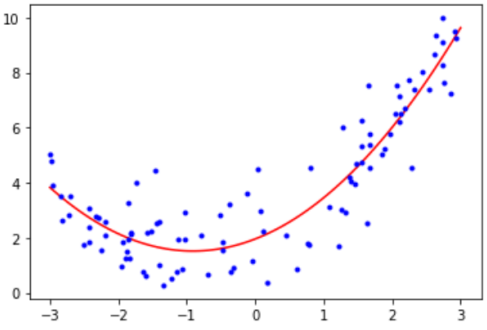
```
array([[0.96672314, 0.53355685]])
```

Polynomial Regression

- Is it good?

```
xs=np.linspace(-3,3,100)
yp=0.53 * xs**2 + 0.967*xs + 1.96
plt.plot(xs,yp,'r');

plt.plot(X,y,'b.');
```

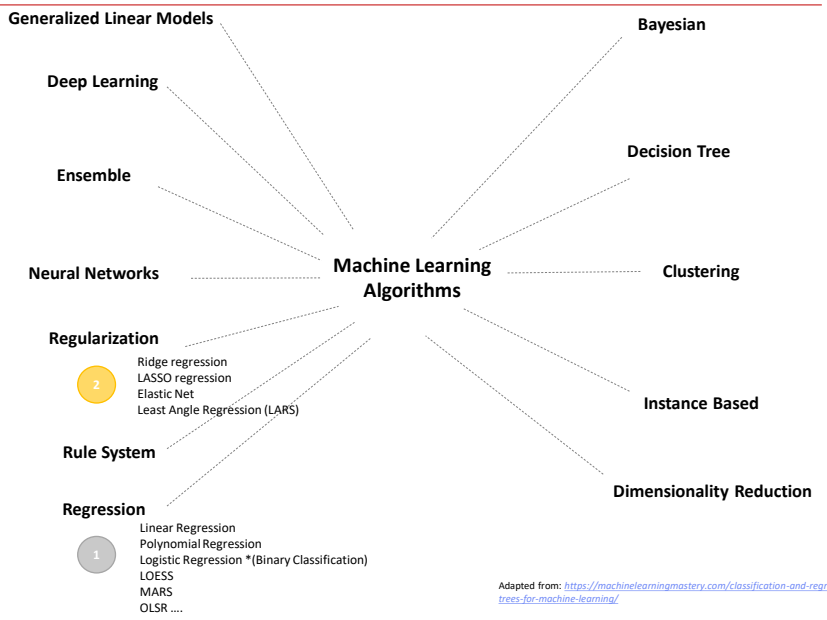


- Good approximation!
- $PolynomialFeatures(degree = d)$ transform an array containing n features into an array containing $\frac{(n+d)!}{d!n!}$ terms
  - Complexity increases!!
  - Usually $d \leq 4$

Regularization

Ridge Regression
Lasso Regression

Machine Learning Algorithms



Adapted from: https://machinelearningmastery.com/classification-and-regression-trees-for-machine-learning/

Ridge/Lasso Regression

- Regularization in machine learning attempts to address the overfitting problem
  - In Regression models, the regularization **constrains/shrinks the coefficient estimates towards zero.**
    - Actually, in regression models **the higher the parameters the higher the risk of overfitting**

- Ridge Regression

Penalty term:
regularizes the coefficients such that if the coefficients take large values the optimization function is penalized.

$$J(\boldsymbol{\theta}) = MSE(\boldsymbol{\theta}) + \alpha \frac{1}{2} \sum_{i=0}^{n} \theta_i^2$$

Adding an additional term to the cost function forces:

1. The model to fit the data
2. At the **same time keep the model's weights as small as possible**

(controlling the risk of overfitting)

Ridge/Lasso Regression

- Lasso Regression

Penalty term

$$J(\boldsymbol{\theta}) = MSE(\boldsymbol{\theta}) + \alpha \sum_{i=0}^{n} |\theta_i|$$

n is the number of weights, α is the penalty constraint

Adding an additional term to the cost function forces:

1. The model to fit the data
2. At the **same time keep the model's weights as small as possible**

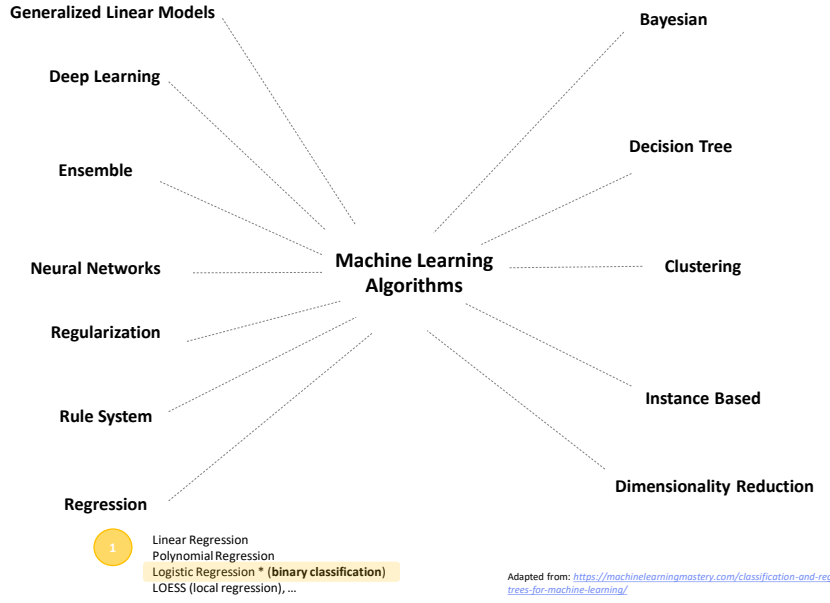
(controlling the risk of overfitting)

- In the case of polynomial regression the reduction of the polynomial degree is a common strategy.

Logistic Regression

Binary Classification

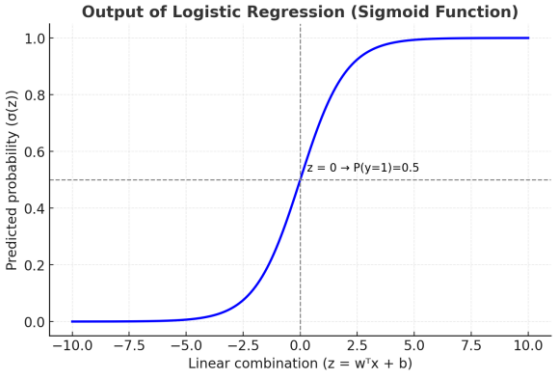
Machine Learning Algorithms



Adapted from: https://machinelearningmastery.com/classification-and-regression-trees-for-machine-learning/

Logistic Regression

- Binary Classifier
  - Classification!!
  - Output (0/1)

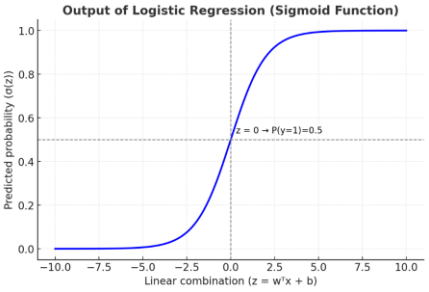


$$\hat{p} = P(y = 1|x) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

$$z = \boldsymbol{\theta}^T \mathbf{x}$$

Logistic Regression

- Logistic regression
  - Applied as binary classifier
  - It is different from linear regression



Linear Regression

In theory, the values in y might assume any value:

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

Logistic Regression

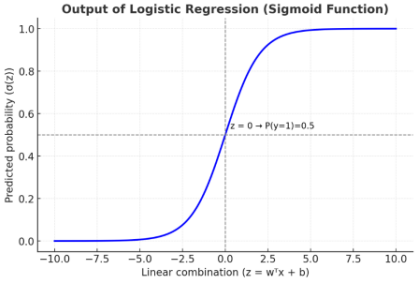
The values in y are probabilities [0,1], determined according to the logistic function:

$$\hat{p} = \frac{1}{1 + e^{-(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n)}}$$

Logistic Regression

- Logistic function
  - Estimates probabilities!
    - Binary Classifier

$$\hat{y} = \begin{cases} 0 & if \quad \hat{p} < 0.5 \\ 1 & if \quad \hat{p} \geq 0.5 \end{cases}$$



- when $z < 0$ then $\sigma(z) < 0.5$
- when $z \geq 0$ then $\sigma(z) \geq 0.5$

$$\hat{p} = P(y = 1|x) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

$$z = \boldsymbol{\theta}^T \mathbf{x}$$

Logistic Regression

- Cost Function
  - The goal is to set $\boldsymbol{\theta}$ such that:
    - The model estimates high probabilities for positive instances ($y = 1$)
    - Low probabilities for negative instances ($y = 0$)

$$\hat{p} = \frac{1}{1 + e^{-(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n)}}$$

  - Contrarily to linear regression, where $\hat{y}$ depends directly from the coefficients θ_i

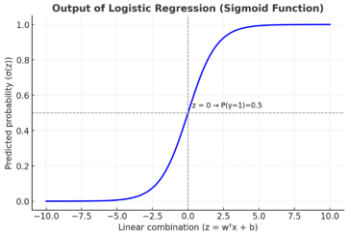
$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

In logistic function:

$$\hat{y} = \begin{cases} 0 & if \quad \hat{p} < 0.5 \\ 1 & if \quad \hat{p} \geq 0.5 \end{cases}$$

Logistic Regression

- Logistic Regression applied to Binary Classification
  - To estimate the probability that a given instance belongs to a given class



$$\hat{y} = \begin{cases} 0 & if \quad \hat{p} < 0.5 \\ 1 & if \quad \hat{p} \geq 0.5 \end{cases}$$

$$\hat{p} = \frac{1}{1 + e^{-(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n)}} \quad \rightarrow \quad \boldsymbol{\theta}\,?$$

Parameters determination
Learning from data

Additional Difficulty

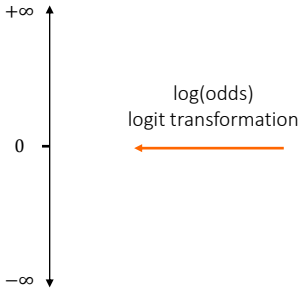
A model's output is limited to $[0,1]$ (probability) no matter the value of $z = \boldsymbol{\theta}^T \mathbf{x}$

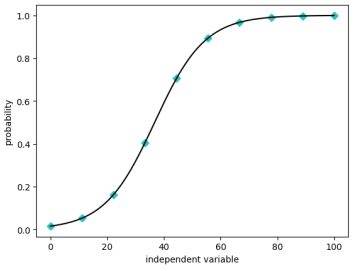
Solution

The logit function, which transforms a probability p $[0,1]$ into a value that ranges from $-\infty$ to $+\infty$.

Logistic Regression

- To calculate $\boldsymbol{\theta}$





Odds

Of a given outcome is the **probability of that outcome occurring** divided by the probability of the **outcome not occurring**

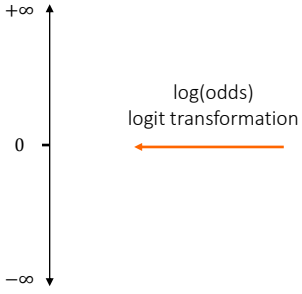
(e.g. an event with 75% of probability has an odds of $\frac{0.75}{0.25} = 3$)

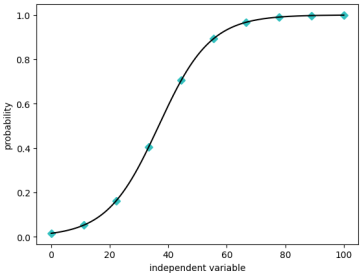
$$odds = \frac{p}{1-p}$$

$$\log(odds) = \ln\left(\frac{p}{1-p}\right) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

Logistic Regression

- To calculate $\boldsymbol{\theta}$



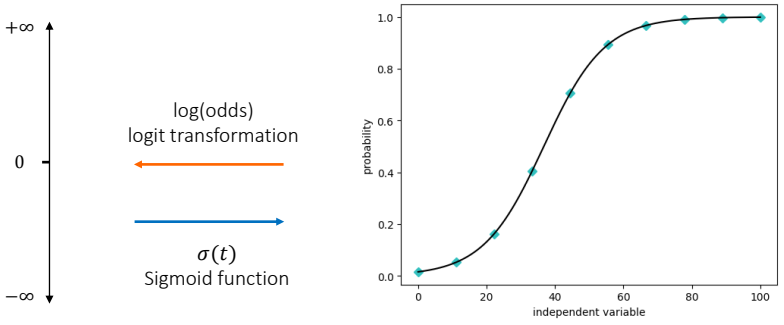


$$logit(p) = \log(odds) = \ln\left(\frac{p}{1-p}\right) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

- The logit transformation expresses the $\log(odds)$ of the probability p as a linear function of the predictors $\boldsymbol{\theta}$, i.e. it connects the linear model to probabilities.
  - The linear predictor $z = \boldsymbol{\theta}^T \mathbf{x}$ (can take any real value) maps to a probability in $[0,1]$.

Logistic Regression

- Mapping log(odds) to probabilities



$$\hat{p} = \frac{e^{(\theta_0+\theta_1 x_1+\theta_2 x_2+\cdots+\theta_n x_n)}}{1+\ e^{(\theta_0+\theta_1 x_1+\theta_2 x_2+\cdots+\theta_n x_n)}} = \frac{1}{1+\ e^{-(\theta_0+\theta_1 x_1+\theta_2 x_2+\cdots+\theta_n x_n)}}$$

Logistic Regression

- Cost Function
  - measures how far the predicted probabilities are from the true labels.
  - The goal is to set $\boldsymbol{\theta}$ such that:
    - The model should estimate: **high probabilities for positive instances** ($y = 1$) and **low probabilities for negative instances** ($y = 0$)

$$\hat{y} = \begin{cases} 0 & if \quad \hat{p} < 0.5 \\ 1 & if \quad \hat{p} \geq 0.5 \end{cases}$$

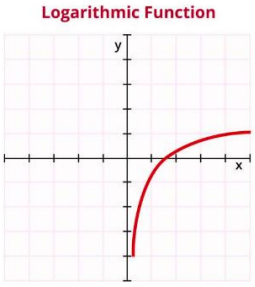
  - The cost function for a **single training instance** is given by:

$$c(\boldsymbol{\theta}) = \begin{cases} -\log(\hat{p}) & if \quad y = 1 \\ -\log(1-\hat{p}) & if \quad y = 0 \end{cases}$$

Logistic Regression

- Cost Function
  - The log function is given by:
    - $y = \log_b(x) \Leftrightarrow x = b^y$

$$\lim_{x \to 0^+} \log_b(x) = -\infty$$



  - For **positive instances** ($y = 1$) :
    - when $\hat{p}$ approaches 0 then $-\log(\hat{p})$ increases
  - For **negative instances** ($y = 0$) :
    - when $\hat{p}$ approaches 1 then $-\log(1 - \hat{p})$ increases

Training: Minimize the cost function!

Logistic Regression

- Cost Function
  - Over the whole training set it is the average cost over all training instances
    - ***log loss function***

$$J(\boldsymbol{\theta}) = -\frac{1}{m}\sum_{1=1}^{m}\left[y^{(i)}\log\left(\hat{p}^{(i)}\right) + (1 - y^{(i)})\log(1 - \hat{p}^{(i)})\right]$$

    - The cost function depends on $\hat{p}^{(i)}$ and $\hat{p}^{(i)}$ depends on $z = \boldsymbol{\theta}^T\mathbf{x}$
  - To minimize this cost function, an optimization method must be applied, like Gradient Descent Method.
    - The partial derivative of the cost function considering θ_j is given by:

$$\frac{\partial}{\partial\theta_j}J(\boldsymbol{\theta}) = \frac{1}{m}\sum_{i=1}^{m}(\sigma\left(\boldsymbol{\theta}^T\mathbf{x}^{(i)}\right) - y^{(i)})x_j^{(i)}$$

Logistic Regression

- The **interpretation** of Logistic Regression coefficients is more intuitive/meaningful when looking at the Odds Ratio:

$$odds = \frac{p}{1-p}$$

$$\log(odds) = \ln\left(\frac{p}{1-p}\right) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

- Example:
  - Predict the probability of an individual suffering from Heart Disease based on two features:
    - Smoking Status (Binary variable; 1 smoker / 0 non smoker)
    - Age (Numeric variable; years).

Logistic Regression

- The fitted Logistic Regression equation gives the following coefficients:

Predictor	Coefficient (θ)	Odds Ratio (e^{θ})
Intercept (θ_0)	-3.5	0.03
Smoking Status (θ_1)	0.85	2.34
Age (θ_2)	0.06	1.06

- Intercept (θ_0)
  - θ_0 = -3.5
    - This is the log odds of having Heart Disease when all predictors are zero (in this case it is unrealistic situation, as age has to be ≠ 0)
    - Odds Ratio (e^{θ}=0.03)
      - It represents the baseline odds of Heart Disease for the reference group (Non-Smoker, Age=0).

Logistic Regression

- The fitted Logistic Regression equation gives the following coefficients:

Predictor	Coefficient (θ)	Odds Ratio (e^{θ})
Intercept (θ_0)	-3.5	0.03
Smoking Status (θ_1)	0.85	2.34
Age (θ_2)	0.06	1.06

- Smoking Status (θ_1)
  - For a one unit increase in Smoking Status the log odds of Heart Disease increases by 0.85, holding Age a constant value.
  - Odds Ratio (e^{θ}=2.34)
    - For the same age, the odds of having Heart Disease for a Smoker are 2.34 times (134% [2.34-1]) higher than the odds for a non Smoker.

Logistic Regression

- The fitted Logistic Regression equation gives the following coefficients:

Predictor	Coefficient (θ)	Odds Ratio (e^{θ})
Intercept (θ_0)	-3.5	0.03
Smoking Status (θ_1)	0.85	2.34
Age (θ_2)	0.06	1.06

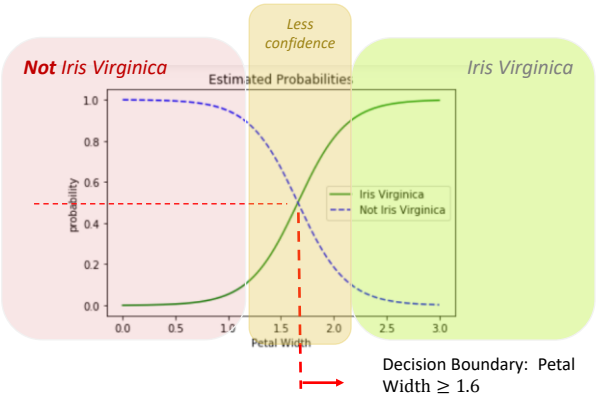
- Age (θ_2)
  - For a one unit increase in Age (years), the log odds of Heart Disease increases by 0.06, holding Smoking Status constant.
  - Odds Ratio (e^{θ} =1.06)
    - For the same Smoking Status, one year increase in Age means that the odds of having Heart Disease multiply by a factor of 1.06 (6%) , i.e. it is associated with a 6% increase in the odds of having Heart Disease.

Logistic Regression

- Problem: Identify Iris Virginica
  - 4 numeric features: (sepal length; sepal width; petal length; petal width)



https://www.datacamp.com/community/tutorials/machine-learning-in-r



$$\hat{y} = \begin{cases} 0 & if \quad \hat{p} < 0.5 \\ 1 & if \quad \hat{p} \geq 0.5 \end{cases}$$

Decision Boundary: Petal Width ≥ 1.6